

Training in PyTorch and Tuning Hyperparameters (Lab 4-2)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, October 7, 2026

Instructor: Prof. Jongmin Lee

How to use this note:

- Read it before the lab session. The lab will not repeat this material.
- Covered: logits, the five calls of one training step, training mode against evaluation mode, early stopping, the three-way split, and grid search. The emphasis is on what each method assumes and where it breaks.
- Not covered: code and function calls. See the docstrings in the skeleton `src/lab04_2.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy (grading and AI use) was given in the policy slides of the first class and is not repeated here.

1 Learning objectives

1. Assemble a model that outputs logits, and explain why the loss applies the sigmoid. (Task 1)
2. Write the five calls of one training step in order, and compute what changes when the gradients are not cleared. (Task 2)
3. Implement early stopping and explain why the best weights are restored. (Task 3)
4. Distinguish which decisions each of the three sets may be used for. (Task 4)
5. Design a hyperparameter grid and pick one on validation alone. (Task 4)

2 Background: what the framework does for you

In Lab 4-1 you wrote the forward and backward passes in numpy yourself. In PyTorch that part shrinks to two lines: `loss.backward()` fills in the gradient of every parameter and `optimizer.step()` updates the values with it. The arithmetic you did by hand in Lab 4-1 is still there, and only the code you write gets shorter.

The Lab 4-2 data is one day of sensor readings from 2,000 factory motors: bearing temperature, vibration, load, rpm, hours since the last service, humidity, and age in months. About 30% of the motors needed unscheduled service within two weeks. A bearing that runs too cold is as dangerous as one that runs too hot, and a single temperature coefficient cannot express that U shape, so a hidden layer earns its place here.

The arithmetic got easier, and now there is more for you to decide. The framework does not pick the learning rate, the hidden size, or how many passes to make, and those values move the result a great deal. A learning rate one factor of ten too small can leave a run at the majority-class score after the whole epoch budget, no better than answering “no service needed” every time; changing only that one number can then move the accuracy by more than twenty points. Table 1 in §3.5 shows the size of the effect on a small problem.

So you run several candidates and pick one, and the Lab 2 problem returns in a subtler form. The best of six comparisons carries luck along with merit, so reporting it as your final performance inflates it. What you build in Lab 4-2 is the procedure that splits the data three ways and tracks which decisions each part may make. Grading follows that procedure.

The four steps of Lab 4-2:

- **Model:** assemble a network that takes seven inputs and returns one logit. (Task 1)
- **One epoch and measurement:** write the five calls of a training step in order, and keep measurement separate from training. (Task 2)
- **Early stopping:** stop when validation loss stops improving, and restore the best weights. (Task 3)
- **Grid search:** compare six configurations on validation alone, and touch the test set exactly once. (Task 4)

3 Key concepts

3.1 The model outputs logits

The Task 1 model is five modules, linear layers and ReLU in alternation, and the last layer produces one number per row. That number is a logit, related to the probability p by $z = \log \frac{p}{1-p}$. Since $z > 0$ is the same as $p > 0.5$, the accuracy computation in Task 2 only checks whether the logit clears zero.

- **Numerical stability:** `BCEWithLogitsLoss` folds the sigmoid and the logarithm into one expression. Apply the sigmoid separately and the logarithm blows up as p approaches 0 or 1.
- **The double-sigmoid accident:** add `nn.Sigmoid()` at the end of the model and the loss applies a second sigmoid to it. No error is raised, and training works badly.

Assumptions and limits:

- A single-logit output is for binary classification only. Too small a hidden size cannot draw the U shape; too large a hidden size fits the training noise.

3.2 One training step: five calls

Updating the parameters once from one mini-batch takes five calls (Figure 1). The order is fixed, and dropping one or moving one trains something else without saying so.

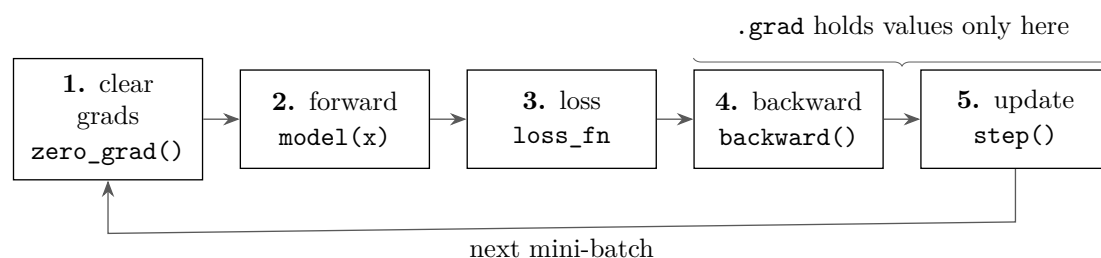


Figure 1. The five calls that train one mini-batch. Call 4 fills in the gradients and call 5 updates the parameters with them. Skip call 1 and the previous batch's gradients are still there, and get added.

What it means that gradients accumulate:

- **Accumulation is the default:** `backward()` adds to `.grad` instead of overwriting it, which is what lets a large batch be processed in pieces.
- **So you clear them every step:** without clearing, the update becomes the sum of the previous batches' gradients, which acts like an arbitrarily inflated learning rate.
- **The wrong position is wrong too:** put `zero_grad()` after `backward()` and you erase the gradients you just computed, then update. §4 computes both cases.

Measurement flips two switches:

- **`model.eval()`:** layers such as dropout and batch normalization stop their training-time behaviour. The Lab 4-2 model has no such layer, so the numbers match either way, but without the habit you will be silently wrong the day you add one.
- **`torch.no_grad()`:** no computation graph is built, which saves memory and time and removes any chance of calling `backward()` by mistake.
- **They are separate switches:** the hidden tests check the training flag and the gradient on the forward output separately, one for each switch.
- **Rounding:** every number these two return is rounded to 6 decimals: a loss from `train_one_epoch`, a loss and an accuracy from `evaluate`.

Assumptions and limits:

- The loss is assumed differentiable in the parameters. Insert an operation with no defined derivative and the gradient stops flowing, with no error and no more learning.
- One batch's gradient only estimates the full one, so small batches update coarsely.

3.3 Early stopping

An epoch is one full pass over the training data. As epochs accumulate the training loss keeps falling while the validation loss stops falling at some point, and past that point the model starts memorizing noise that exists only in the training data. Early stopping halts training when the validation loss has not improved for a fixed number of epochs, and that number is the patience (Figure 2).

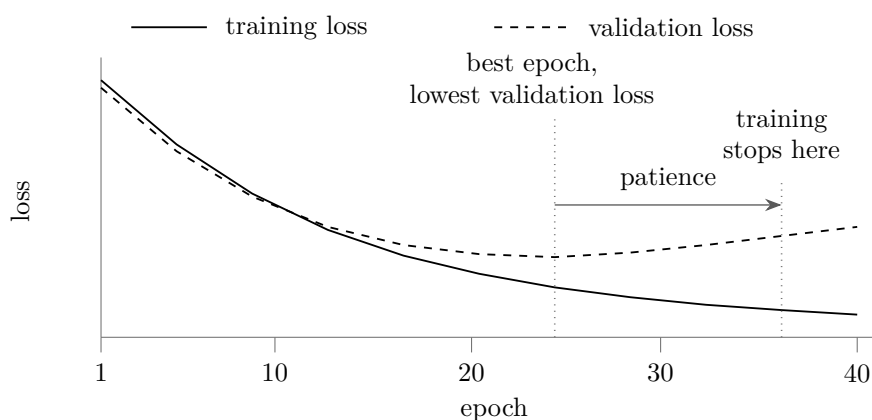


Figure 2. Learning curves and where early stopping fires. A schematic, not measured data. The validation loss bottoms out while the training loss keeps falling; training stops once **patience** epochs have passed with no new best, and the weights of the best epoch are the ones returned. Your own curve for Report Figure 1 will be noisier.

Spec (Task 3)

Compute the validation loss every epoch. If it is **strictly smaller** than the smallest value so far, record it as the best, remember the epoch, save a copy of the weights, and reset the count of epochs without improvement to 0. Otherwise add 1 to that count, and stop when it reaches **patience**. After the loop, restore the saved best weights into the model.

What to know:

- **One bad epoch is not a stop:** the patience separates wobble from a real turn.
- **The weights at the stopping epoch are not the best ones:** the last patience epochs are by definition a stretch without improvement. With no saved copy you hand in a model you deliberately made worse.
- **The stopping epoch is the best epoch plus the patience:** this holds unless the budget runs out, and the hidden tests check the identity.

Assumptions and limits:

- The rule assumes the validation loss is a fair signal. It is measured on a few hundred rows, so a short flat stretch can end a run that would still have improved. A longer patience trades compute for that risk.
- Stopping early is one way to limit overfitting, not the only one, and it changes nothing about the model's capacity. Weight decay and a smaller network act on the cause instead.

3.4 The three-way split: which set decides what

Lab 2 split the data two ways. Lab 4-2 compares six configurations and also picks the epoch to stop on, so the set used for picking is no longer unseen data (Figure 3).

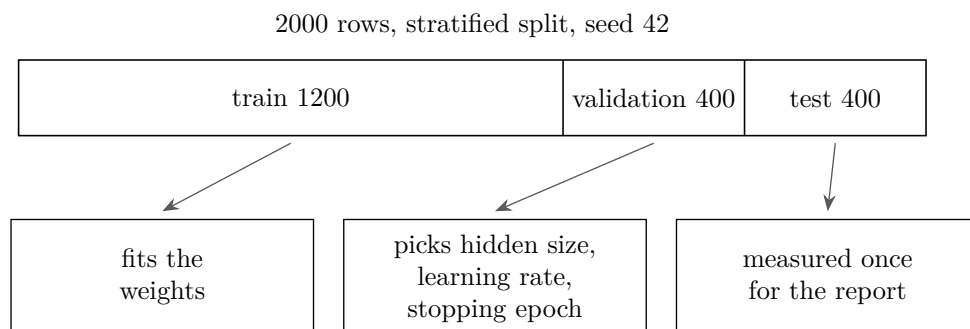


Figure 3. The three sets and the decisions each may make. Which set a decision was made on is what fixes the meaning of the number. The test set has one arrow, and that arrow is drawn only after the search is over.

Why the validation score is inflated:

- Picking the best of six mixes real performance with luck on those 400 validation rows, and because you took a maximum, the luck leans favourable.
- The more candidates and the smaller the validation set, the larger the bias.

- Used once, the test number took part in no decision and carries no such bias. Used twice, it has become a second validation set.

Assumptions and limits:

- The three sets are assumed to come from the same distribution. An accuracy from 400 rows has sampling error, so a third-decimal difference settles nothing.

3.5 Designing a grid search

A hyperparameter is a value you set rather than one that training finds. Lab 4-2 grid-searches the learning rate against the hidden size. Table 1 is that same search on a *different, smaller* problem (700 rows, four features, hidden sizes 4 and 16) run through the code you are about to write. Task 4 runs the lab's own grid on the lab's data, and you read those six rows off `results.json`.

Table 1. A 2 x 3 grid on a toy problem. Not the lab's data: 700 rows, 140 validation rows whose majority-class accuracy is 0.6500, and the same budget the lab uses (80 epochs, patience 10), which is why the rows that never improve stop at 80. The learning rates are spaced by factors of ten, and the learning rate moves the result far more than the hidden size does.

hidden size	learning rate	val. loss	val. accuracy	best epoch
4	0.01	0.6360	0.6500	80
4	0.1	0.4685	0.7571	80
4	1.0	0.4558	0.7857	17
16	0.01	0.6483	0.6500	80
16	0.1	0.1418	0.9643	80
16	1.0	0.1177	0.9429	29

Design rules:

- **Space learning rates on a log scale:** 0.01, 0.1, 1.0, by multiples. The gap between 0.01 and 0.02 usually drowns in noise; the gap between 0.01 and 0.1 is as clear as the top two rows of Table 1, where the loss falls by about a quarter and the accuracy rises by ten points.
- **Change one thing at a time:** `set_seed()` is called again for every configuration, so within one hidden size the three learning rates share the same initial weights and the same batch order.
- **Give every configuration the same budget:** fix the maximum epochs and the patience. Give one side more epochs and you cannot say what won.
- **Fix the selection criterion in advance:** Task 4 takes the smallest validation loss. In Table 1 that picks the last row, even though the row above it has the higher accuracy (0.9643 against 0.9429). Accuracy is a step function of whether the logit clears zero and wobbles easily, while the loss also reflects how confident each prediction was. Deciding afterwards which of the two numbers to quote is how a search turns into wishful thinking.

Assumptions and limits:

- An interaction between the two values is missed unless the grid is fine enough.
- **Across hidden sizes the comparison is less controlled than within one.** The weight shapes differ, so the initial values cannot match, and building the larger model draws more random numbers, which shifts the batch order too.
- **A winner at the edge of the grid means the grid must be widened.** In Table 1 the winner has the largest hidden size and the largest learning rate on offer, and both sit at the edge of the grid, so neither can be called optimal until a wider grid has been tried.

4 Worked examples

Example 1. Two steps by hand:

Take the simplest model, one input and one weight, with $w = 0.50$, $b = 0.00$, a learning rate of 1.0, and `BCEWithLogitsLoss` averaged over the batch; Table 2 holds the two mini-batches. The logit is $z = wx + b$ and the probability is $p = \sigma(z)$, and for the logistic loss the derivative with respect to the logit reduces to $p - y$, so the gradients of the batch-mean loss are $\partial L / \partial w = \frac{1}{n} \sum_i (p_i - y_i) x_i$ and $\partial L / \partial b = \frac{1}{n} \sum_i (p_i - y_i)$.

Table 2. The two mini-batches of the worked example. Two rows in each batch. The same two batches are processed in the right order and in the wrong one, and the results are compared.

	batch 1		batch 2	
input x	1.0	2.0	3.0	-1.0
label y	1	0	1	0

1. **Batch 1:** $z = (0.5, 1.0)$, $p = (0.6225, 0.7311)$, loss = 0.8937, $p - y = (-0.3775, 0.7311)$.
2. **Batch 1, gradient and update:** $\partial L / \partial w = 0.5423$ and $\partial L / \partial b = 0.1768$, so $w \leftarrow -0.0423$, $b \leftarrow -0.1768$.
3. **Batch 2:** $z = (-0.3036, -0.1345)$, $p = (0.4247, 0.4664)$, loss = 0.7423.
4. **Batch 2, gradient and update:** $\partial L / \partial w = -1.0962$ and $\partial L / \partial b = -0.0544$, so $w \leftarrow 1.0539$, $b \leftarrow -0.1223$.

Without clearing the gradients:

- Batch 1's gradient is still in `.grad` when batch 2 is backpropagated, so the two are added: $0.5423 + (-1.0962) = -0.5539$.
- The update uses that sum, giving $w \leftarrow 0.5116$ and $b \leftarrow -0.2991$, already 0.5423 away from the correct 1.0539 after two steps.
- Both runs print the same losses, 0.8937 and 0.7423, so the output alone does not reveal it.

Example 2. When does early stopping fire:

Suppose `patience = 3` and the validation losses in Table 3.

Table 3. Validation loss per epoch and the count of epochs without improvement. The point is that epoch 6 sets a new minimum and resets the count to 0. Stopping at epoch 5 would have missed that value.

epoch	1	2	3	4	5	6	7	8	9
validation loss	0.62	0.55	0.49	0.47	0.48	0.46	0.47	0.47	0.49
best so far	0.62	0.55	0.49	0.47	0.47	0.46	0.46	0.46	0.46
epochs without improvement	0	0	0	0	1	0	1	2	3

1. The lowest validation loss is 0.46 at epoch 6, so `best_epoch = 6`.
2. Epochs 7, 8, and 9 bring no improvement, so the count reaches 3 and training stops at epoch 9. `epochs_run = 9 = 6 + 3`, and the weights returned are the copy from **epoch 6**.
3. A rule that stopped on the first increase would have stopped at epoch 5 and missed the minimum.

Where this note and the spec appear to disagree, **the spec governs**.

5 Common mistakes

1. **Not clearing the gradients, or clearing them after backward.** As in §4, only the parameters differ: no error, and a plausible printed loss.
2. **Leaving out `eval()` or `no_grad()` in the measurement function.** The numbers look the same, and the hidden tests catch it from the gradient on the forward output.
3. **Dividing the loss by the number of batches.** The last batch is usually smaller, and the spec asks for a per-sample mean.
4. **Returning the weights of the last epoch.** Without the restore you submit the weights from the stretch where nothing improved.
5. **Not reseeding for each configuration.** You cannot tell a learning-rate effect from luck in the initial weights. Suspect this when two runs disagree.
6. **Selecting a configuration on the test set.** The hidden tests count how many times the test set is measured.

6 How this maps to the lab tasks

Table 4. Tasks matched to concept sections. Which section each task draws on, and what grading looks at. Worth checking once before you write the report.

Lab task	Concept	What grading looks at
Task 1. The model	§3.1	the five specified modules, one logit out
Task 2. Epoch and eval	§3.2, §4	call order, eval mode, no grad
Task 3. Early stopping	§3.3, §4	patience handling, best weights restored
Task 4. Grid search	§3.4, §3.5	selection on validation, one test pass
Report figures 1–3	§3.3, §3.5	curves, grid, validation against test

7 Self-check

1. Why does code that never clears the gradients raise neither an error nor a warning?
2. Validation loss rose once at epoch 5 and then hit a new minimum at epoch 6. With a patience of 3, at which epoch does training stop, and whose weights are returned?
3. You picked among six configurations on the test set. Which way is that number biased?
4. The winning hidden size in Table 1 sits at the edge of the grid. What do you do next?

These questions are fair game for the oral check: you must explain every line you submit.

8 Further reading

- PyTorch Docs, Autograd mechanics
<https://pytorch.org/docs/stable/notes/autograd.html>
- PyTorch Docs, `torch.optim`
<https://pytorch.org/docs/stable/optim.html>
- Goodfellow, Bengio & Courville, *Deep Learning*, §7.8 and §11.4
<https://www.deeplearningbook.org/>